Лабораторна робота №6

Мета роботи: Сформувати практичні навички реалізації логування в ROS2, навчитися здійснювати моніторинг стану системи, виявляти помилки та реалізовувати механізм повернення до стабільної версії (rollback).

Посилання на репозиторій: Anastasia11070830/ROS2

Хід роботи

1. Працюємо в тій самій віртуальній машині що і раніше.
2. Активація ROS2-середовища, а також перевірка нод і топіків.

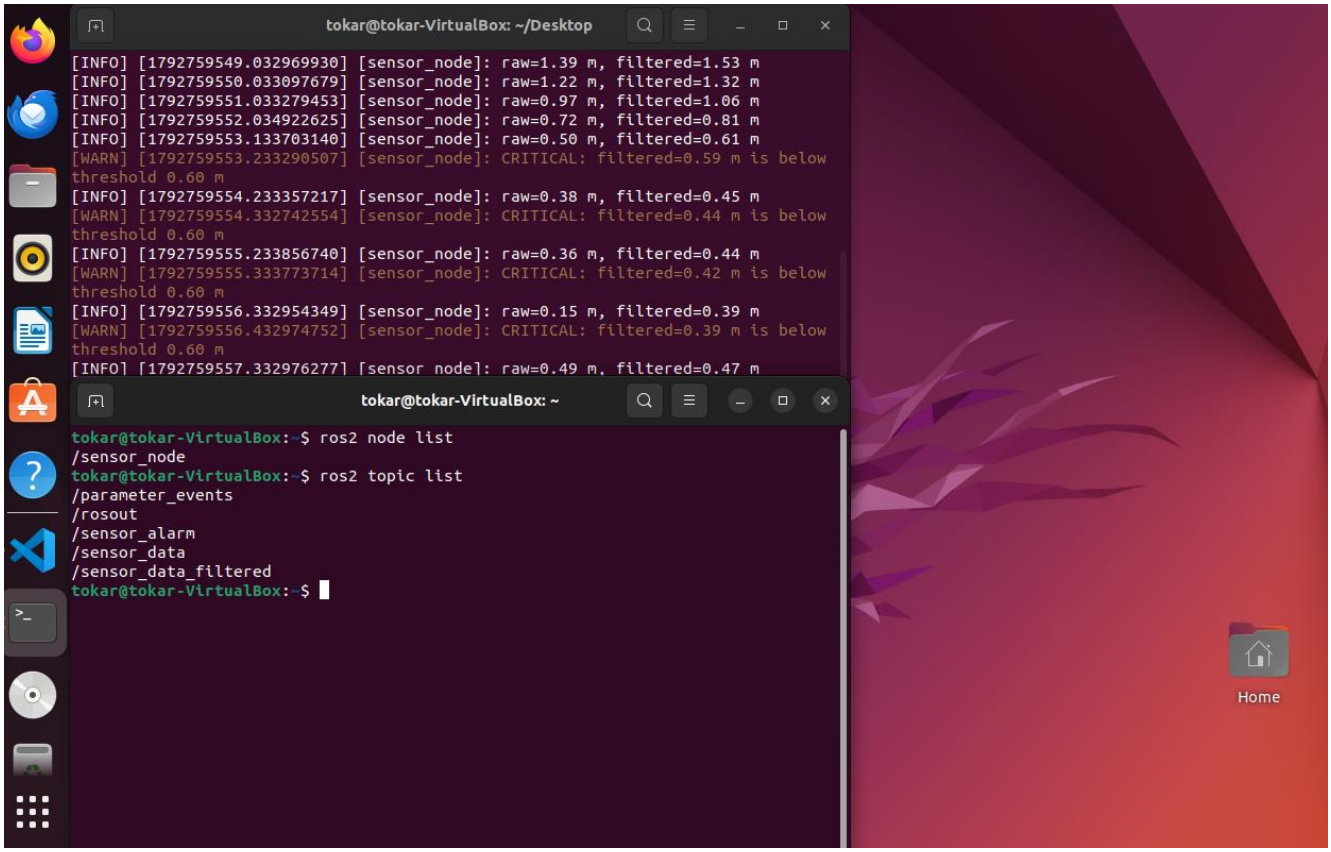


Рисунок 1 – демонстрація термінала 1 та 2 з виконанням

3. **Реалізація логування.** Відкриваємо проєкт у VS Code. Пишем новий код який має навчити вузол ROS 2 писати логи (повідомлення про помилки, дебаг), імітувати збої та подавати сигнал "серцебиття" (heartbeat).

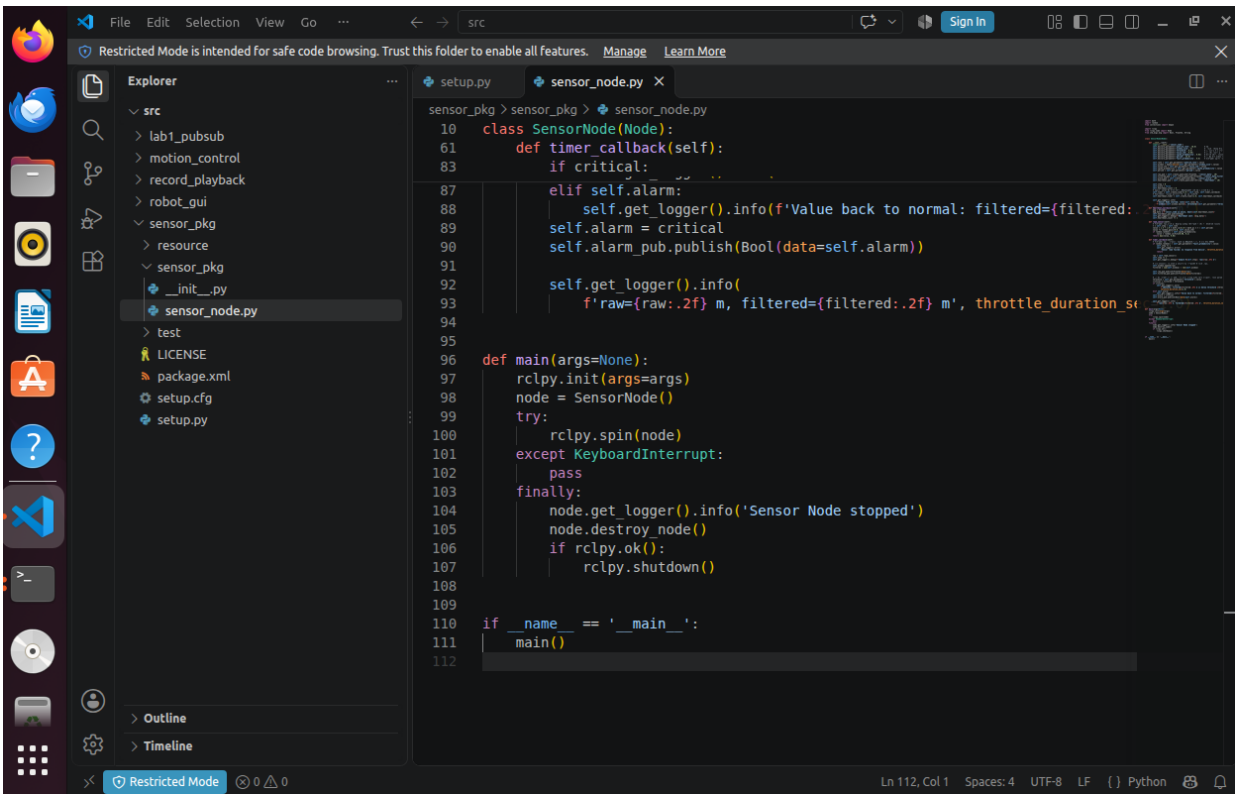


Рисунок 2 – проєкт у VS Code

4. Перевірка роботи логування (INFO, WARN, ERROR) шляхом динамічної зміни параметрів вузла під час його роботи.

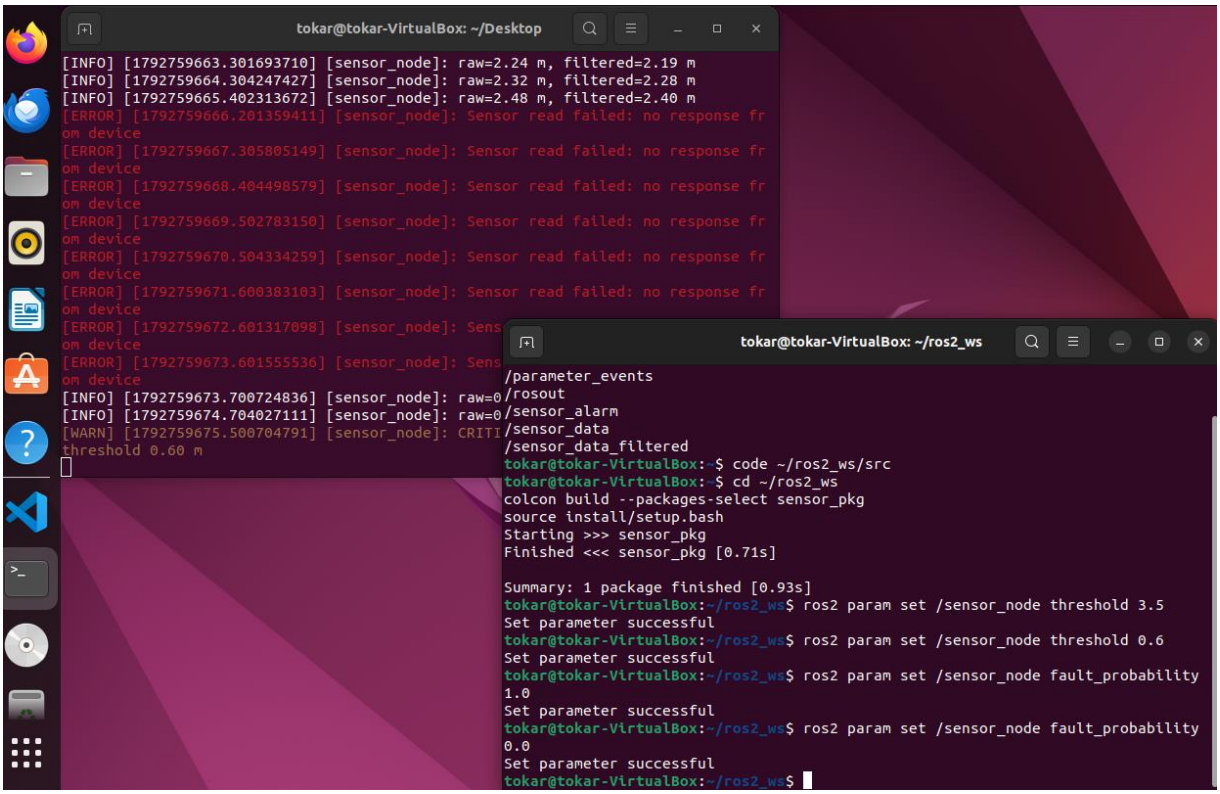


Рисунок 3 – доведення, що система вміє правильно реагувати на події та змінювати рівень тривожності логів залежно від заданих параметрів.

5. **Реалізація heartbeat.** Доводимо, що вузол працює стабільно та генерує сигнали через задані проміжки часу, за допомогою сигналу heartbeat (серцебиття).

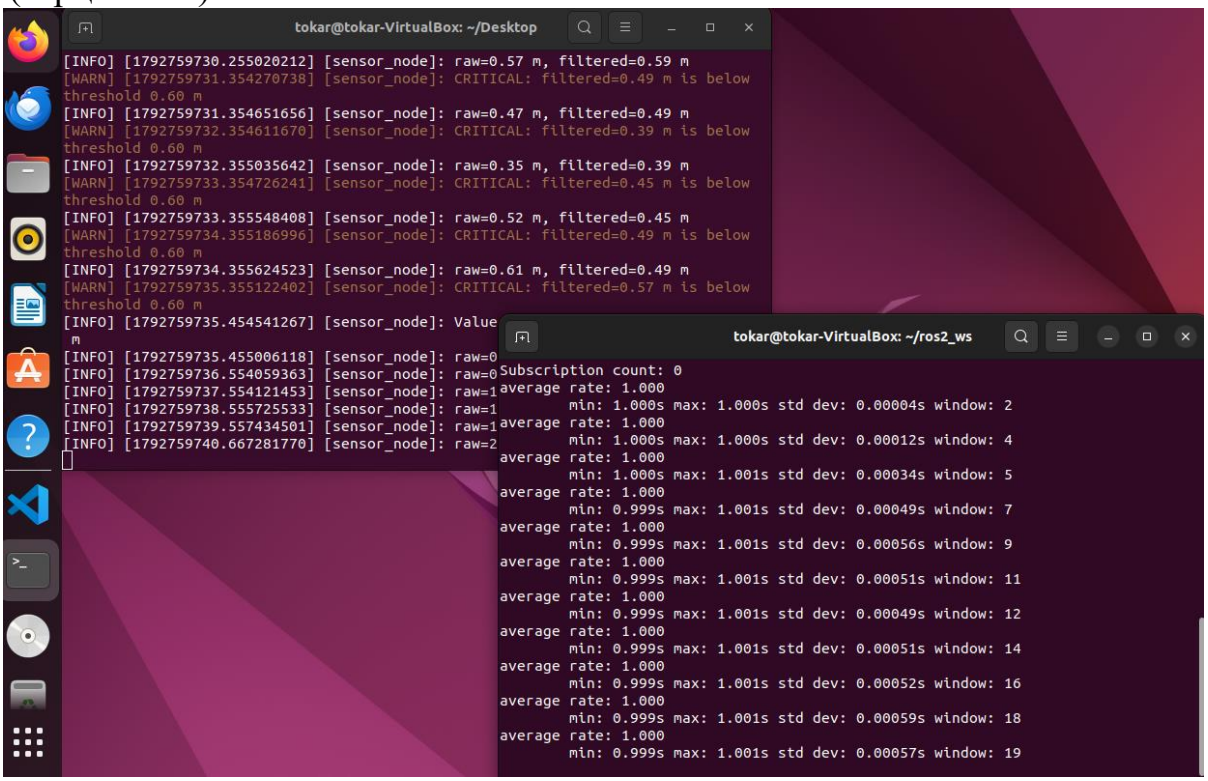


Рисунок 4 –перевірка працездатності.

6. **Моніторинг системи.** Створюємо новий порожній пакет для майбутнього вузла моніторингу.

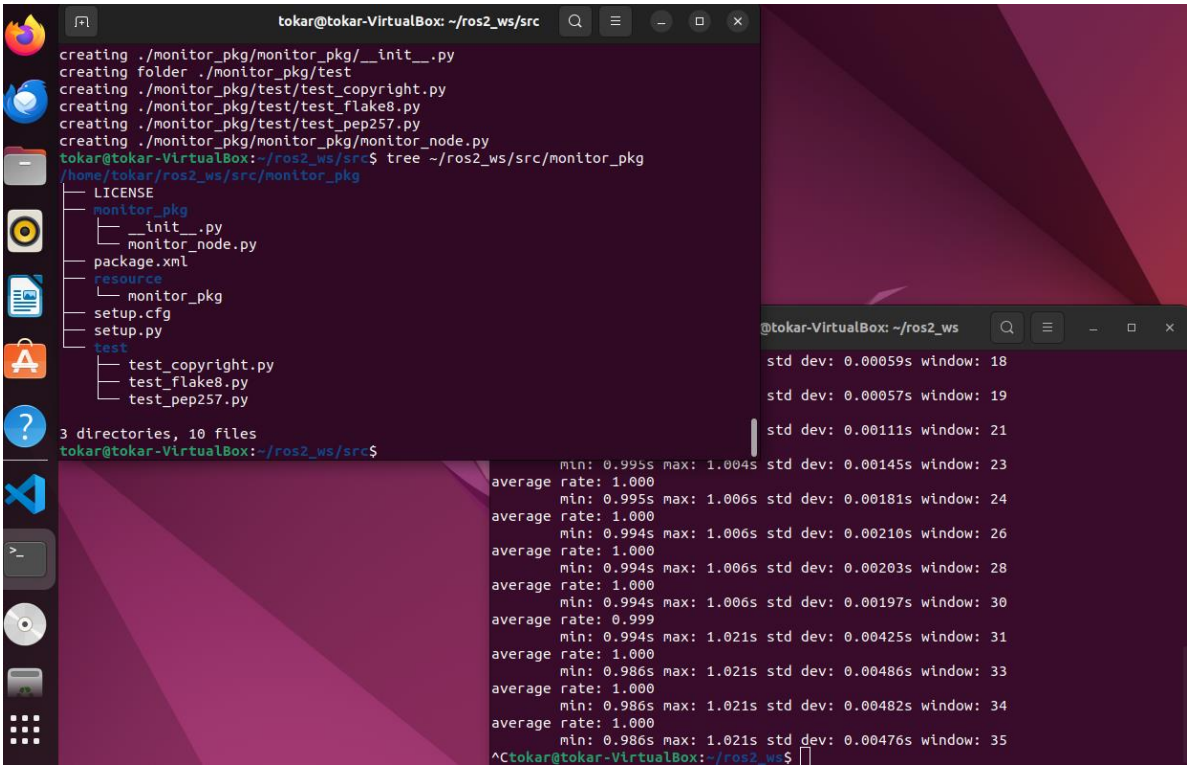
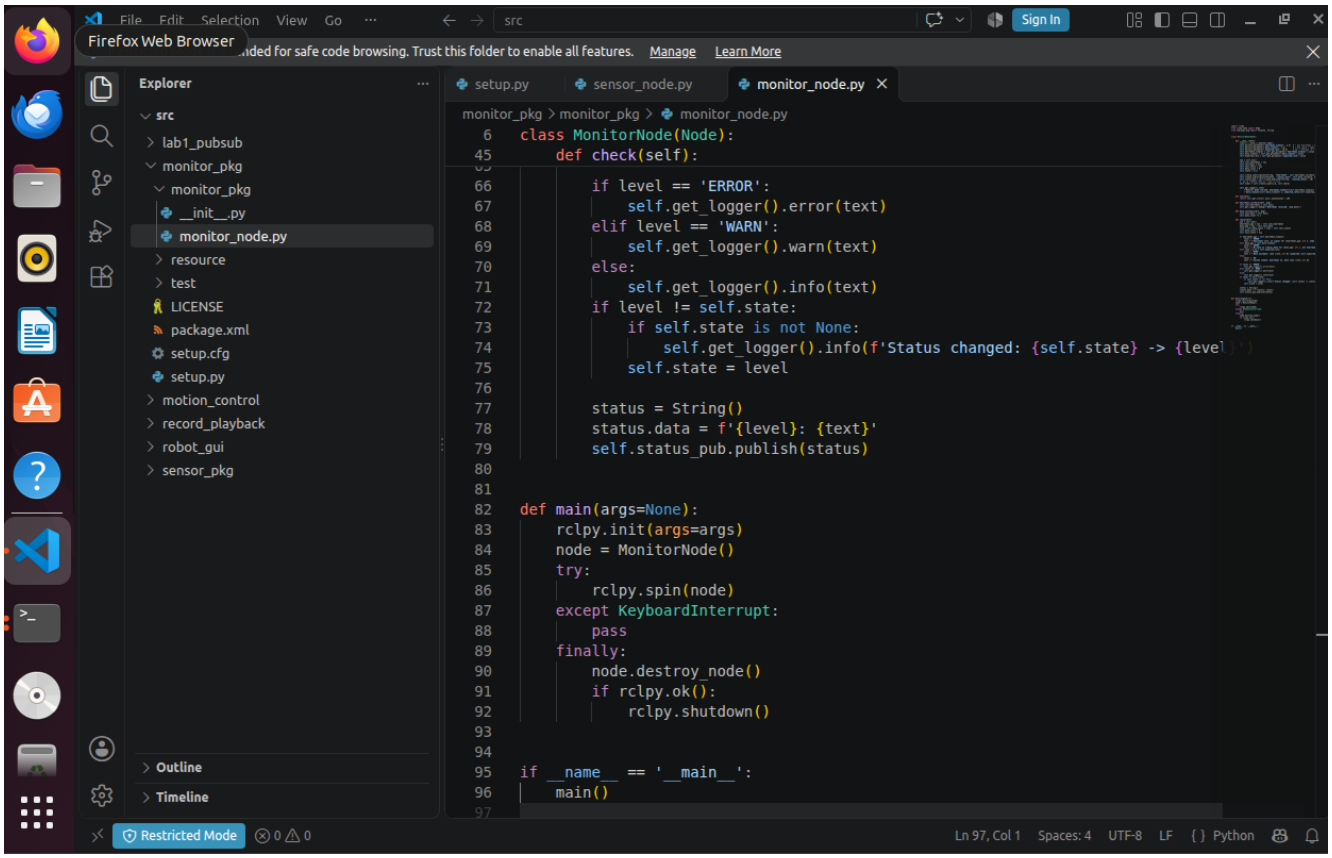


Рисунок 5 –моніторинг системи

7. Задаємо параметри для вузла моніторингу (monitor_node), який буде слідкувати за порядком у системі. Тепер цей вузол готовий приймати сигнали від інших частин системи і повідомляти про помилки, якщо щось піде не так.




```
class MonitorNode(Node):
    def check(self):
        if level == 'ERROR':
            self.get_logger().error(text)
        elif level == 'WARN':
            self.get_logger().warn(text)
        else:
            self.get_logger().info(text)
        if level != self.state:
            if self.state is not None:
                self.get_logger().info(f'Status changed: {self.state} -> {level}')
            self.state = level

        status = String()
        status.data = f'{level}: {text}'
        self.status_pub.publish(status)


def main(args=None):
    rclpy.init(args=args)
    node = MonitorNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Рисунок 6 – Демонстрація коду вVS code

8. Фінальна збірка та демонстрація того, як "Сенсор" і "Монітор" працюють разом.

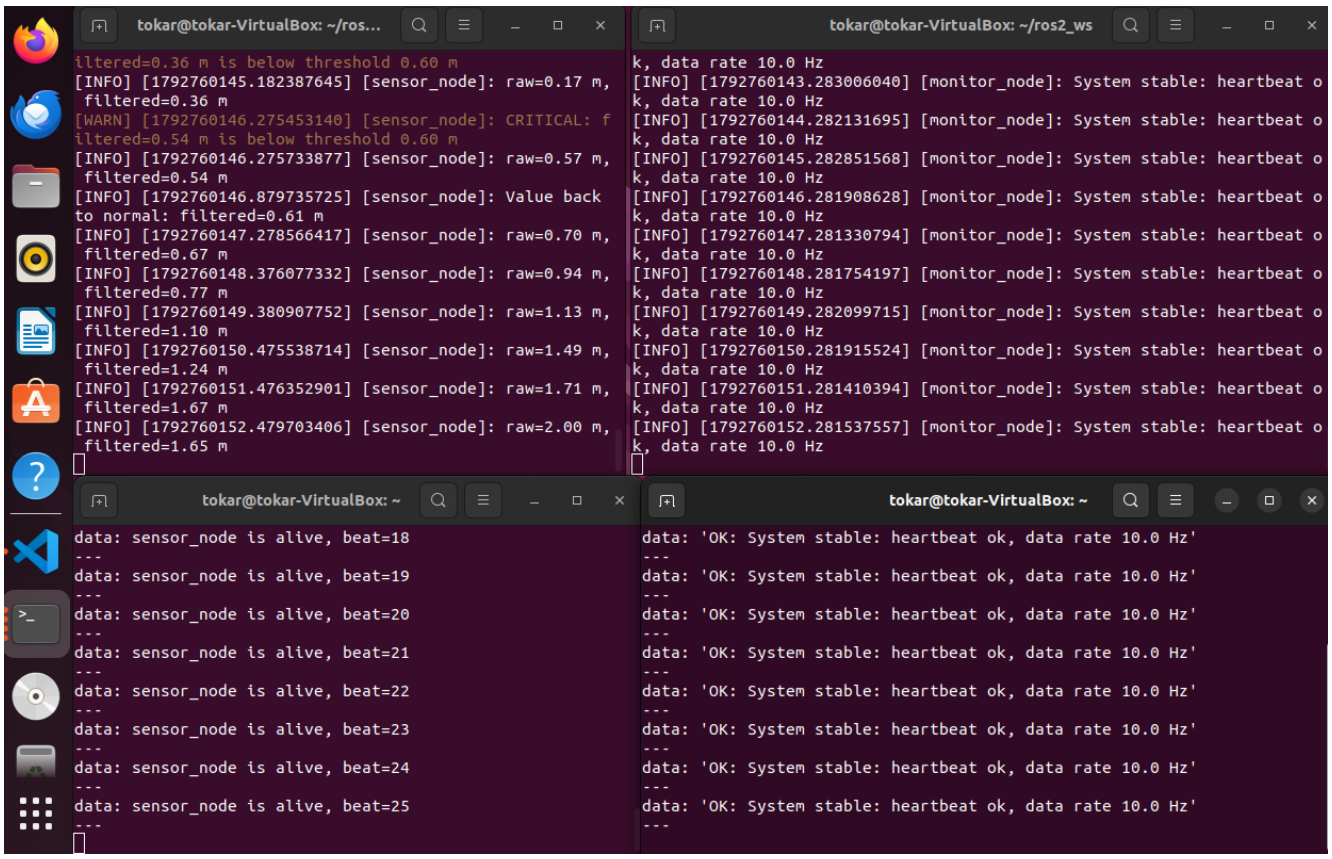


Рисунок 7 – Демонстрація System stable: heartbeat ok, data rate 10.0 Hz

9. Перевіряємо як система моніторингу реагує на погіршення та повну відмову даних від сенсора.

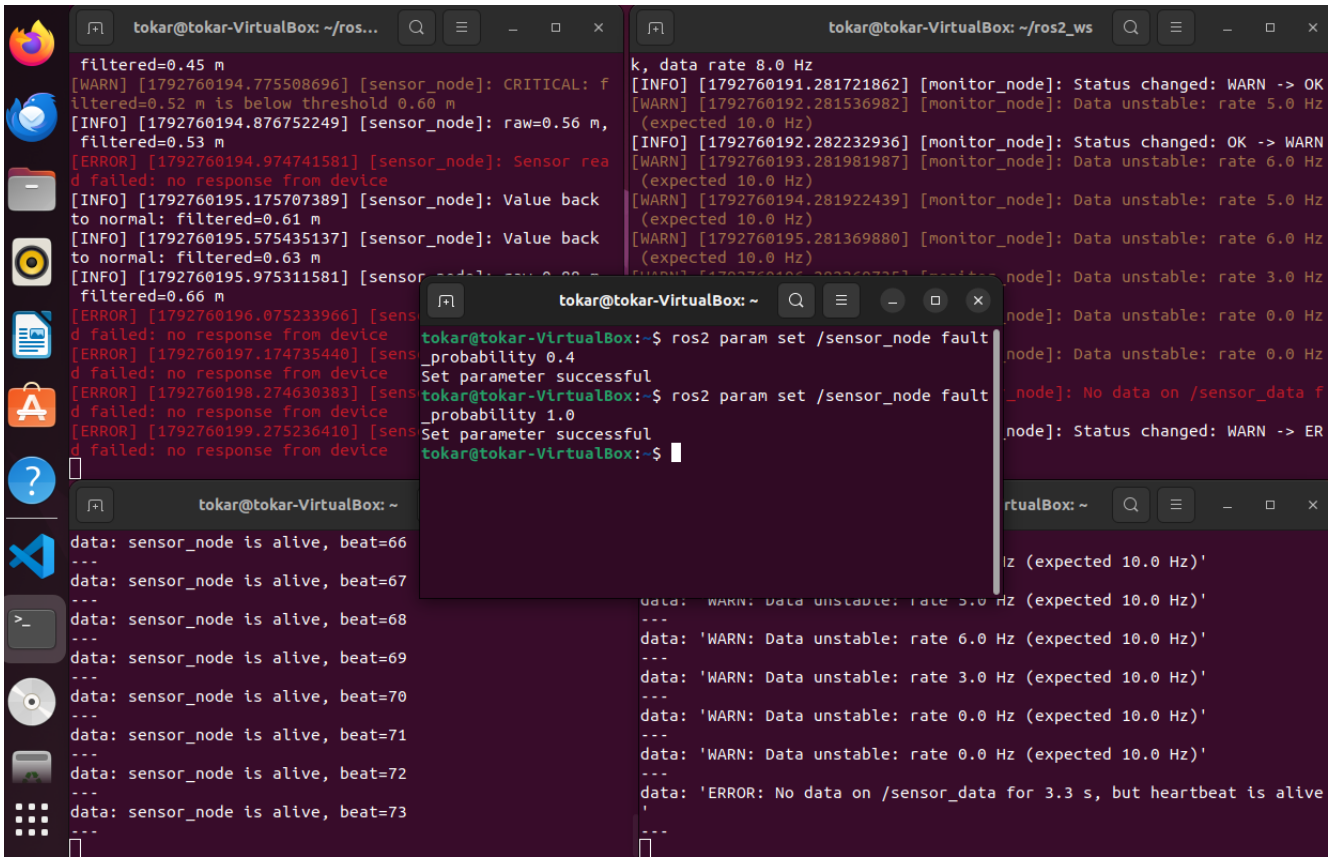


Рисунок 8 – Демонстрація що система розрізняє ситуації "сенсор видає погані дані" (WARN) та "сенсор не видає даних взагалі" (ERROR)

10. Імітація повного "падіння" (смерті) вузла сенсора, щоб перевірити, чи помітить це монітор.

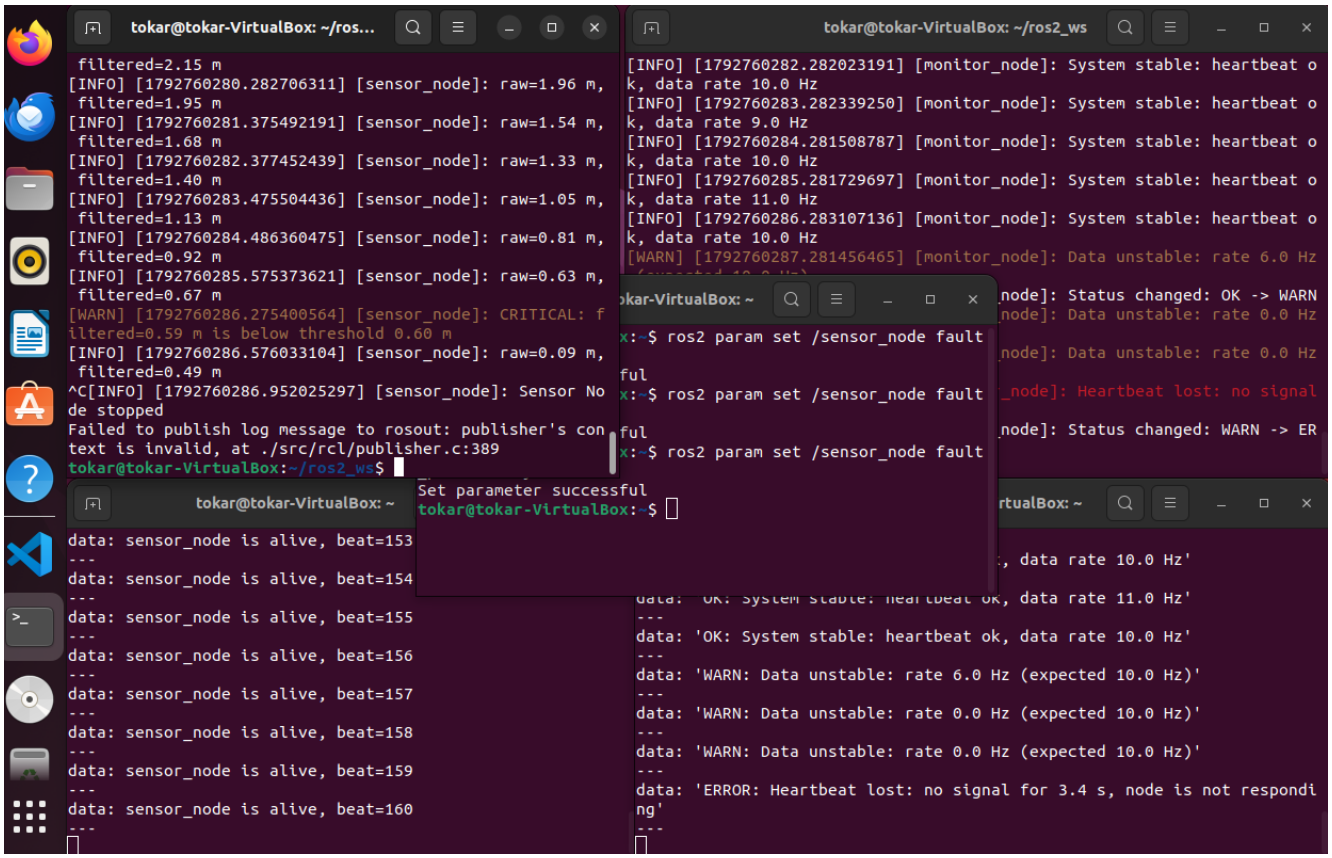


Рисунок 9 – Доводимо, що довели, що система моніторингу розуміє різницю між "сенсор видає погані дані" та "сенсор повністю вимкнувся/зламався".

11. Робимо збереження поточного стану всього проєкту за допомогою системи контролю версій Git.

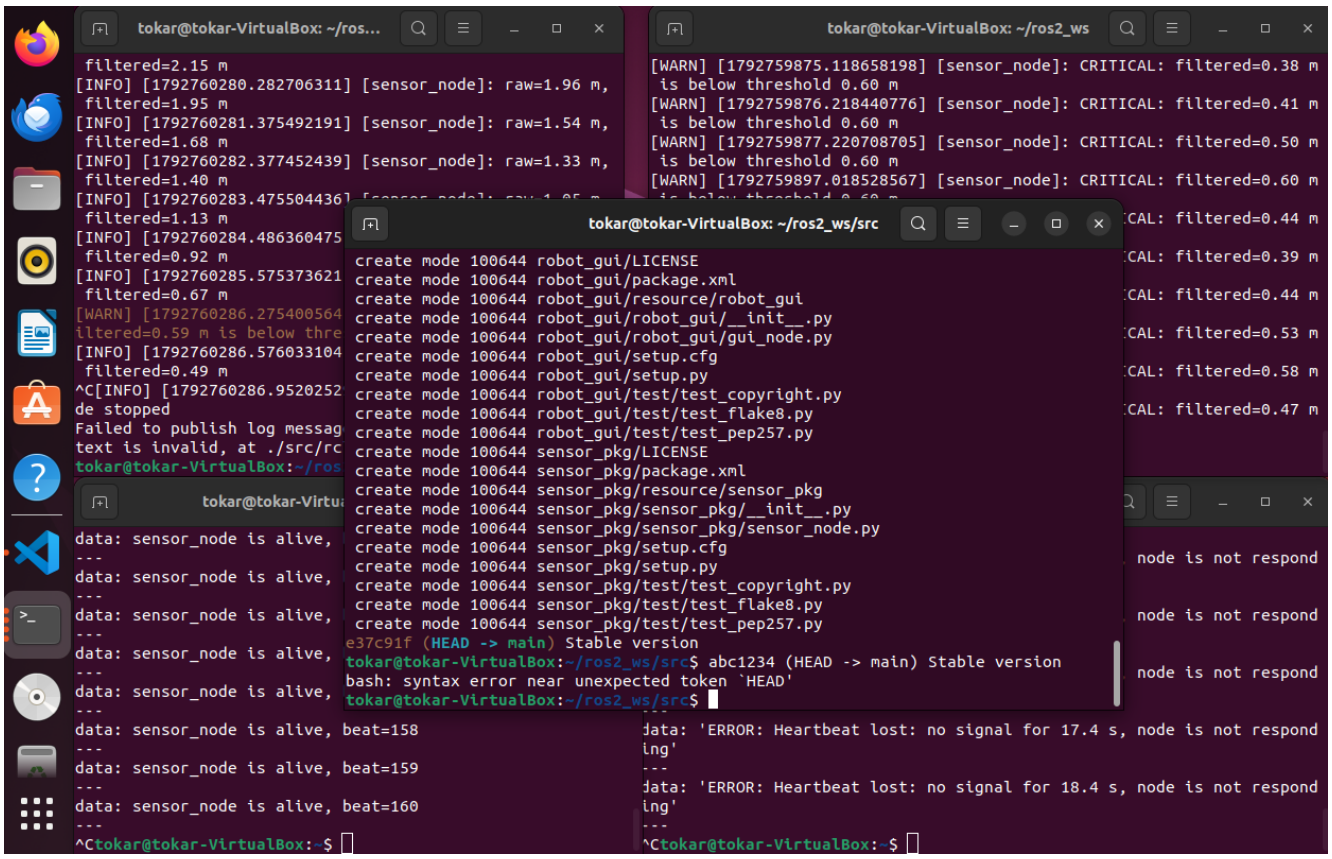


Рисунок 10- Робота з GIT

12. **Реалізація rollback.** Робимо навмисне внесення помилки в код та перевірка цієї зміни за допомогою Git.

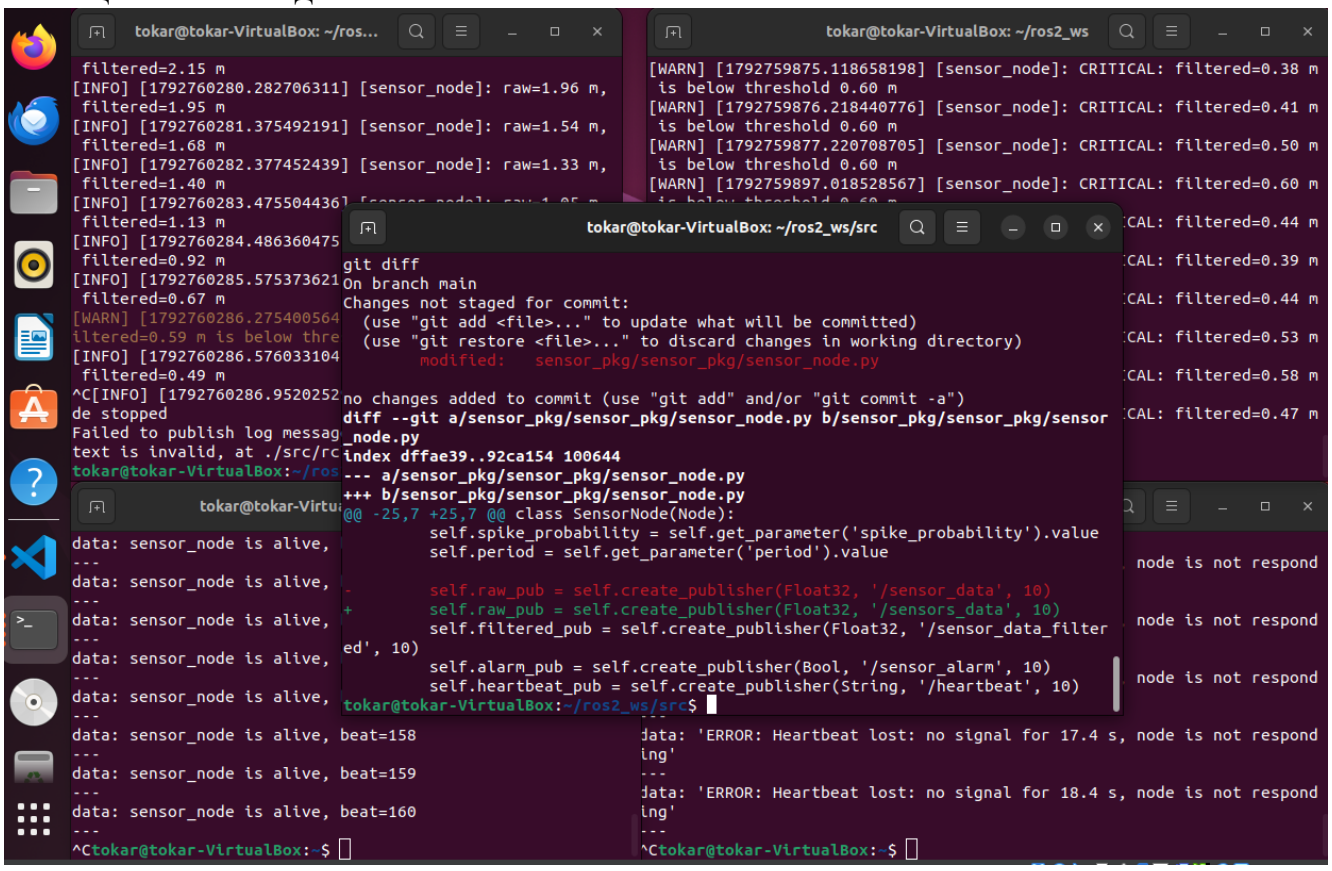


Рисунок 11- Демонстрація того як за допомогою Git можна легко "відкотити" (rollback) цю помилку назад до стабільної версії.

13. **Переконатися, що система працює некоректно.** Переконалися, що навіть одна зайва літера в назві топіка повністю ламає зв'язок між програмами, хоча вони обидві продовжують "жити".

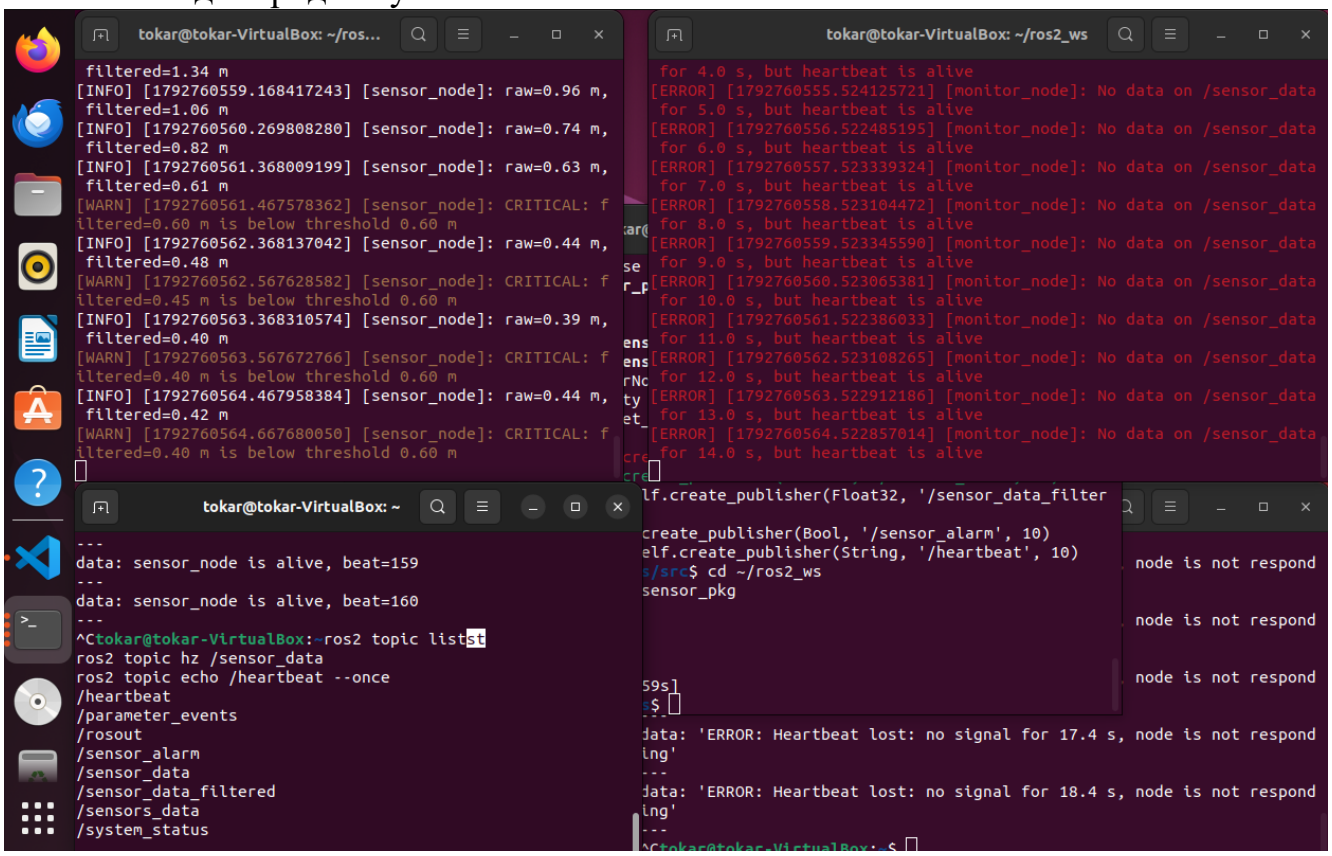


Рисунок 12 – Демонстрація того що система працює некоректно

14. **Фіксація помилкової версії в Git.** Демонстрація того, як за допомогою Git "відкотити" (скасувати) погані зміни назад до стабільної версії.

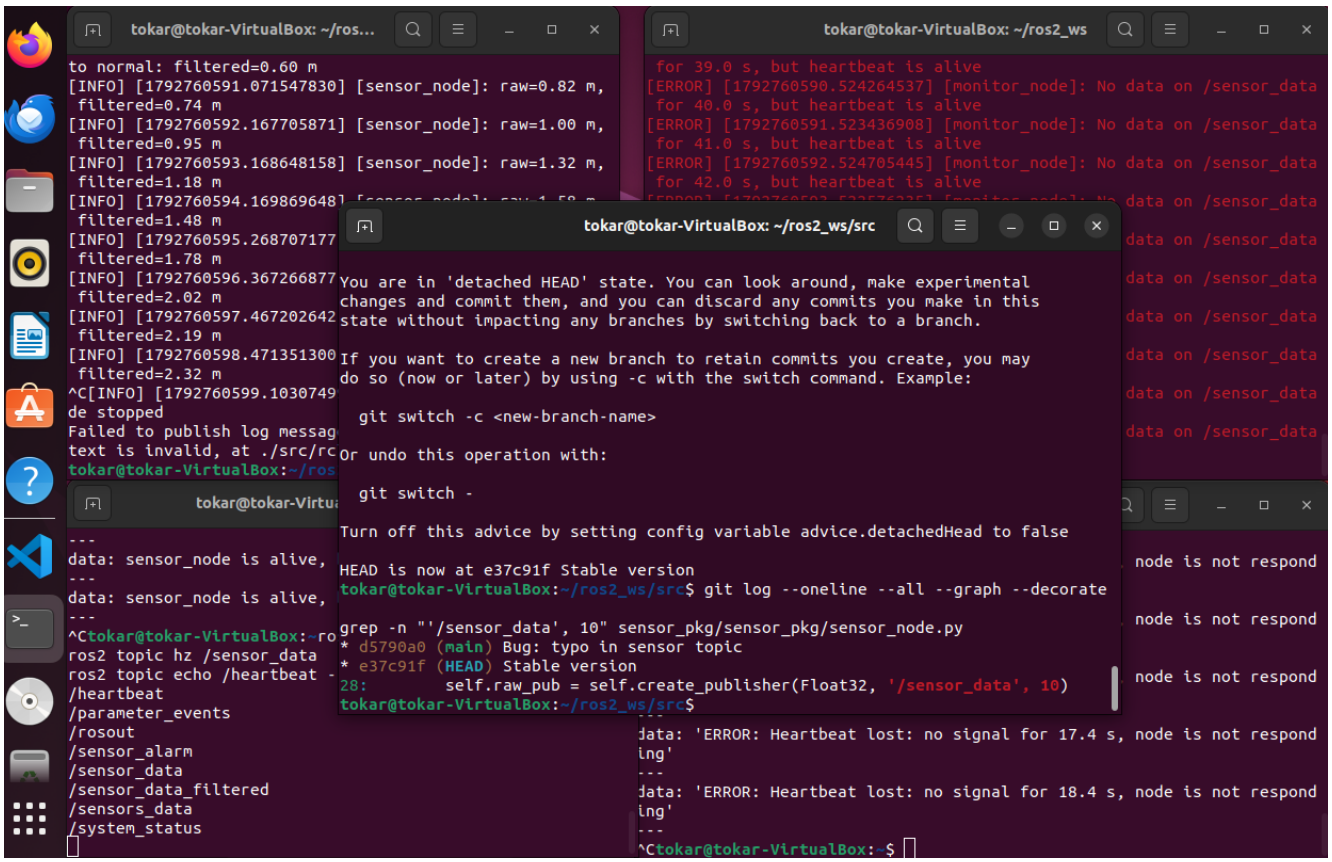


Рисунок 12 – Демонстрація того як скасувати зміни

15. **Перевірка відновлення роботи**. Робимо успішне підтвердження того, що після "відкату" (rollback) помилки система знову працює ідеально.

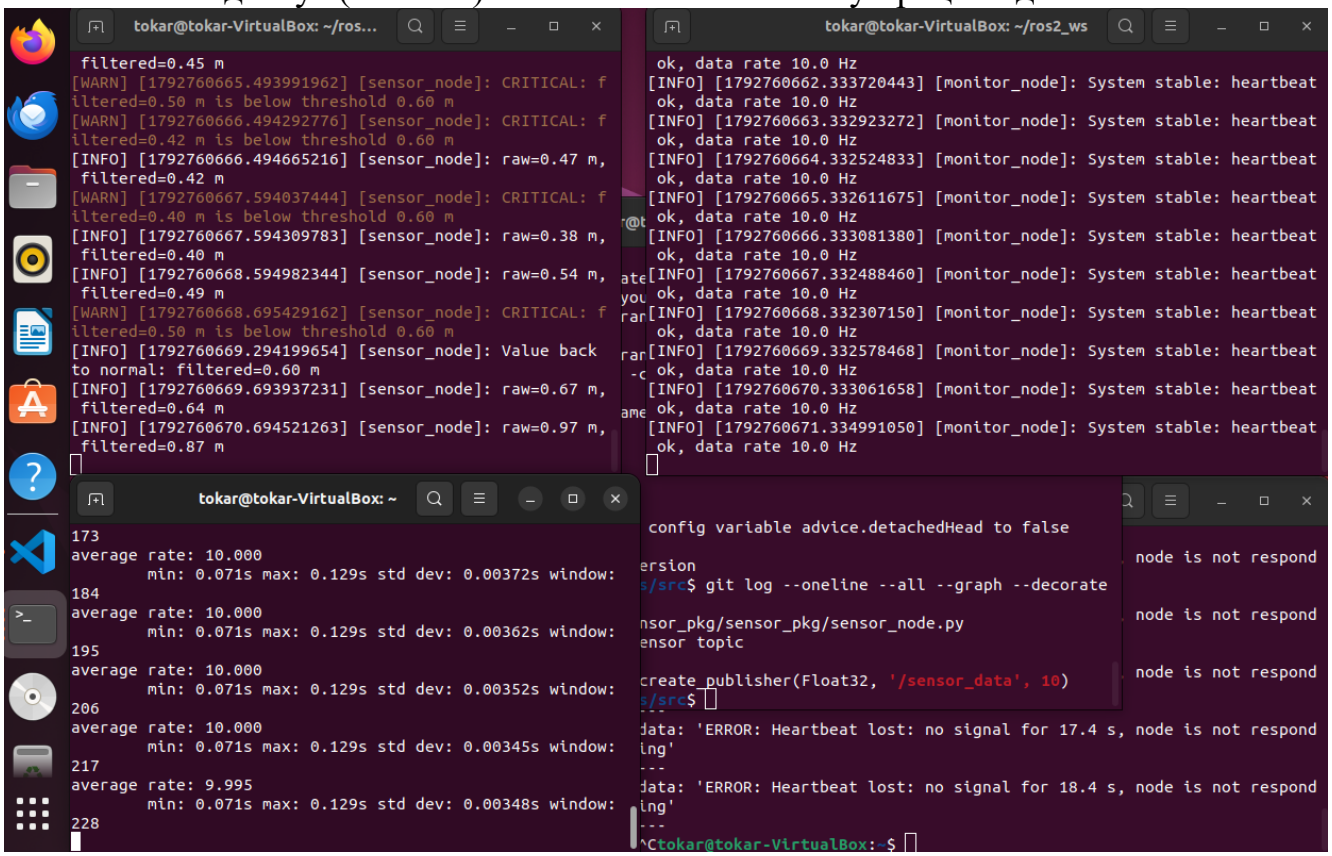


Рисунок 13 - За допомогою однієї команди, скасували навмисну помилку і зв'язок між сенсором та монітором повністю відновився.

16. **Повернення на гілку `main` і закріплення відкату.** Робимо правильне, "безпечне" скасування помилки в Git без переписування історії та збереження доказів для звіту**.**

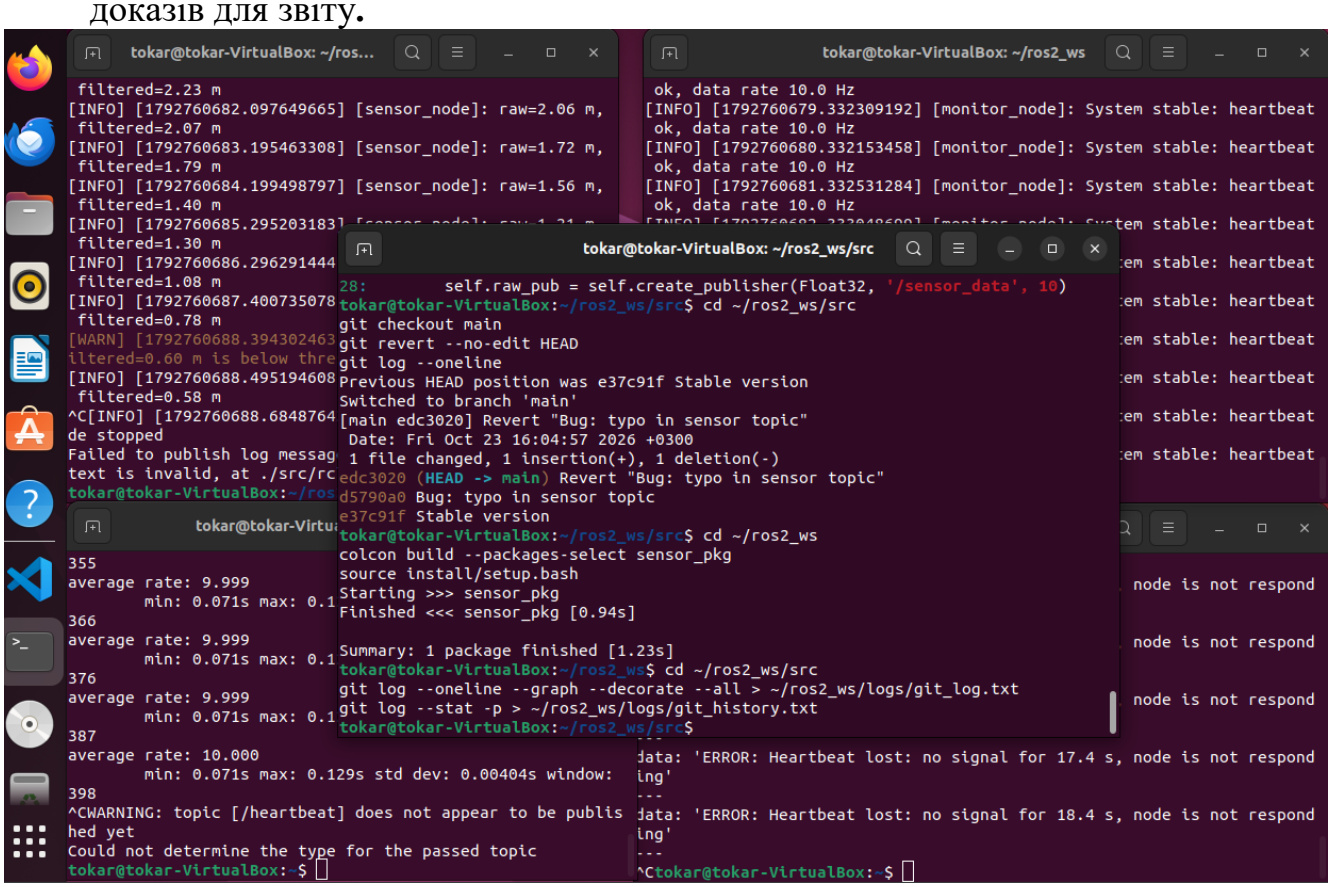


Рисунок 14 – Завершення роботи Git

Висновок: У ході виконання лабораторної роботи було розроблено та протестовано систему моніторингу для вузлів ROS 2, а також відпрацьовано практичні навички використання системи контролю версій Git для забезпечення надійності програмного коду. На початку роботи було вдосконалено вузол сенсора (sensor_node), до якого додали механізм генерації сигналу «серцебиття» (heartbeat) з частотою 1 Гц та налаштували різні рівні логування (INFO, WARN, ERROR), що дозволило візуально відстежувати стан системи та імітувати аварійні ситуації. Далі було створено окремий пакет із вузлом моніторингу (monitor_node), який успішно відстежує працездатність сенсора. За допомогою нього було протестовано реакцію системи на різні стани: стабільну роботу з правильним потоком даних (10 Гц), нестабільність (виникнення попереджень WARN при падінні частоти), відсутність даних при живому вузлі (помилка ERROR) та повну відмову вузла (втрата heartbeat). Окремим важливим етапом стала робота з Git: після збереження стабільної версії проєкту (коміт "Stable version") було штучно внесено помилку (друкарську описку в назві топіка), що призвело до розриву зв'язку між сенсором і

монітором. За допомогою команд Git (checkout та revert) було успішно продемонстровано процес безпечного відкату змін (rollback) до стабільного стану без втрати історії комітів, а результати збережено у вигляді логів. Загалом, лабораторна робота довела, що поєднання внутрішнього логування, зовнішнього моніторингу через топіки та використання систем контролю версій є критично важливим для швидкого виявлення, діагностики та усунення несправностей у сучасних робототехнічних системах.